



CURSO EN YOUTUBE

LangGraph desde Cero: construye una Aplicación de IA Real

Nodos • Aristas • Memoria • Human in the Loop • Proyecto final



NODOS

Unidades
inteligentes



ARISTAS

Conexiones
y flujos



MEMORIA

Contexto que
potencia



**HUMAN
IN THE LOOP**

Intervención
humana



**PROYECTO
FINAL**

App de IA
completa



Aprende creando.

Construye tu propia aplicación de IA con **LangGraph**.

```
{
  "nodes": [
    {
      "id": "agente",
      "type": "llm"
    }
  ],
  "edges": [
    {
      "from": "inicio",
      "to": "agente"
    }
  ]
}
```

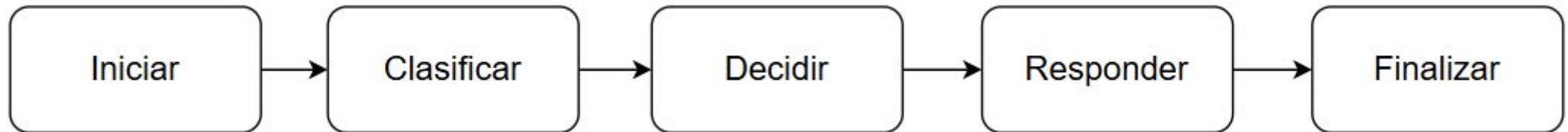


Agenda

- LangGraph
- Grafos
- Estados
- Nodos
- Hands-on Lab

Qué problema resuelve LangGraph

LangGraph nos ayuda a orquestar workflows y agentes de IA usando grafos. Esto significa que podemos definir pasos, conexiones, decisiones y estado compartido entre las partes del flujo.



Qué es LangGraph

LangGraph es un framework para construir aplicaciones de IA basadas en grafos, donde cada paso del flujo se representa como un nodo y la información compartida viaja dentro de un estado.



LangGraph

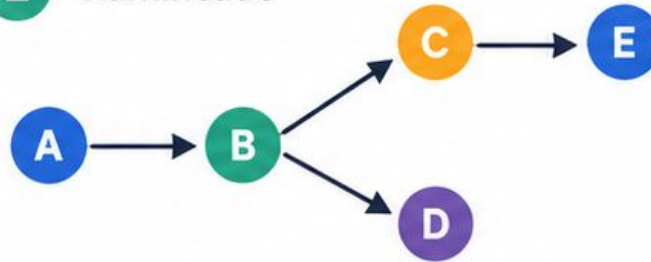
Qué es un grafo

1 Lineal



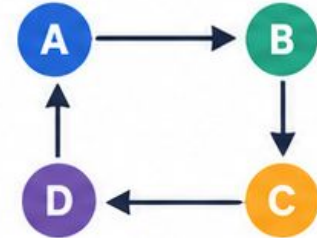
Flujo secuencial dirigido de inicio a fin.

2 Ramificado



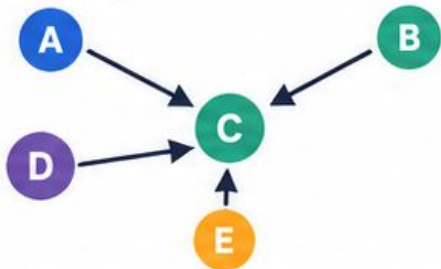
Un nodo se divide en múltiples caminos dirigidos.

3 Cíclico



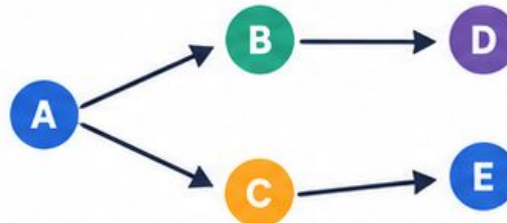
El flujo puede regresar al punto de inicio mediante un ciclo dirigido.

4 Convergente



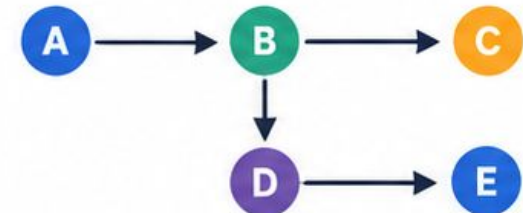
Varios caminos dirigidos convergen en un nodo central.

5 Dirigido acíclico



Un grafo dirigido sin ciclos permite flujos ordenados.

6 Dirigido con rutas alternativas



Las conexiones dirigidas permiten definir rutas alternativas.

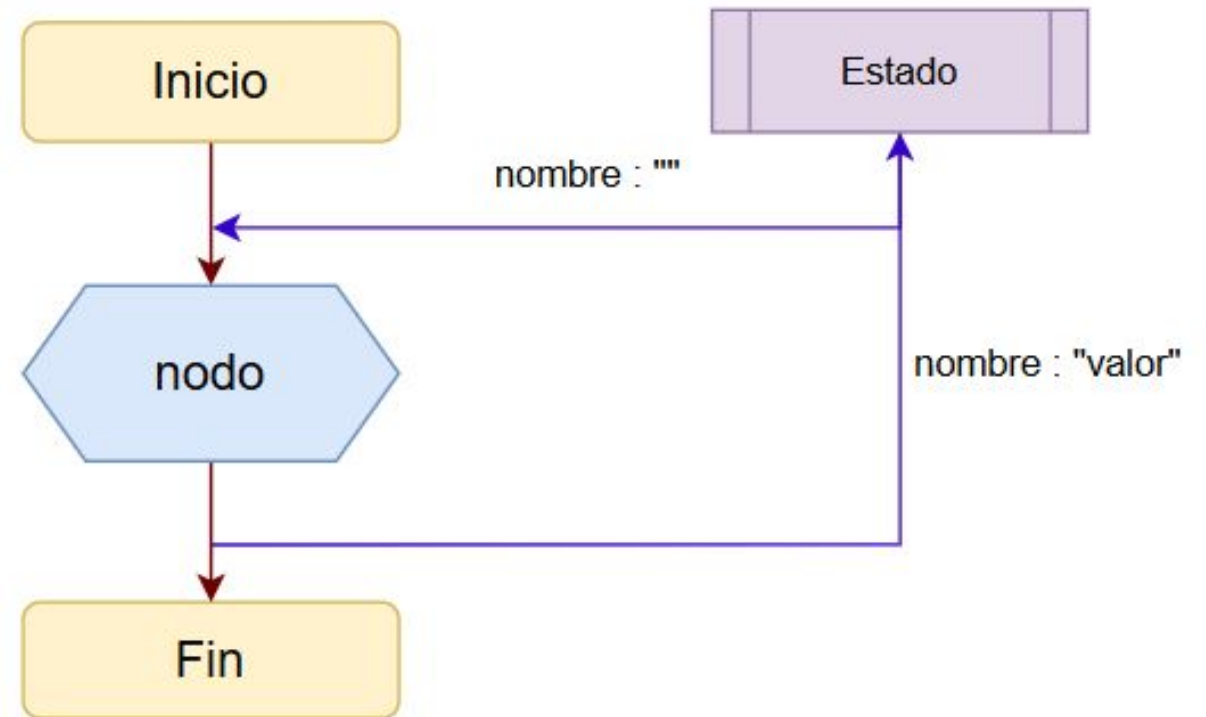
Qué es un grafo en LangGraph

Grafo = conjunto de nodos conectados por aristas

Nodo = paso o responsabilidad

Arista = conexión entre pasos

Estado = información que viaja durante la ejecución



Qué es el estado en LangGraph

El estado es la estructura que transporta la información durante la ejecución del grafo. Cada nodo puede leer ese estado, procesarlo y devolver una actualización del estado.

```
class State(TypedDict):  
    messages: list
```

En este primer ejemplo, nuestro estado tendrá una sola clave: **messages**. Ahí guardaremos los mensajes de la conversación.

El problema: ¿cómo se actualiza el estado?

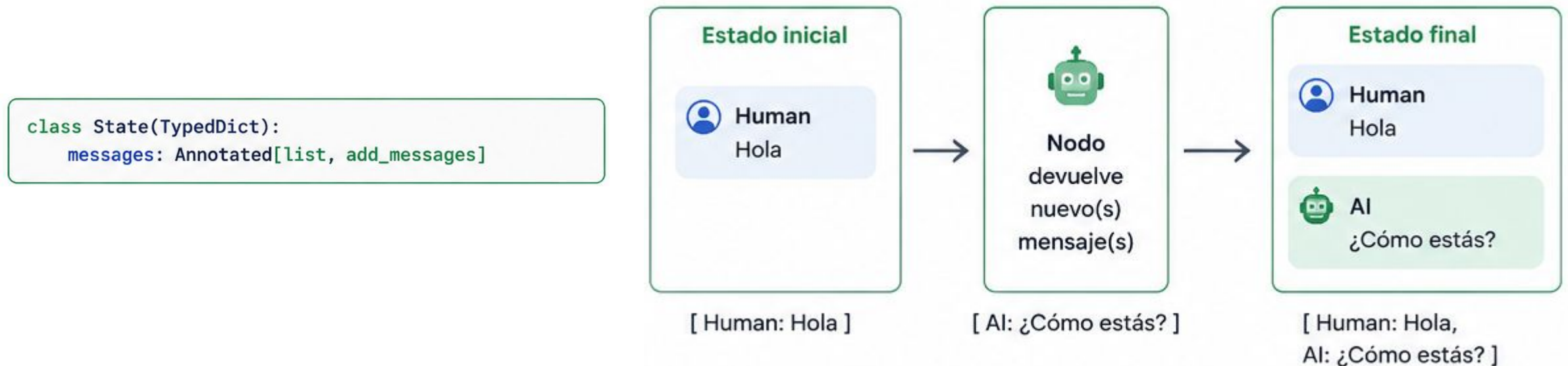
Cuando usamos solo list, LangGraph no sabe que queremos conservar el historial. Por defecto, cada actualización sobrescribe el valor anterior del estado.



El estado final queda solo con el nuevo mensaje, se pierde el historial anterior

El problema: ¿cómo se actualiza el estado?

LangGraph permite definir cómo debe actualizarse un campo del estado usando **reducers**. Para conversaciones, se usa **add_messages**.



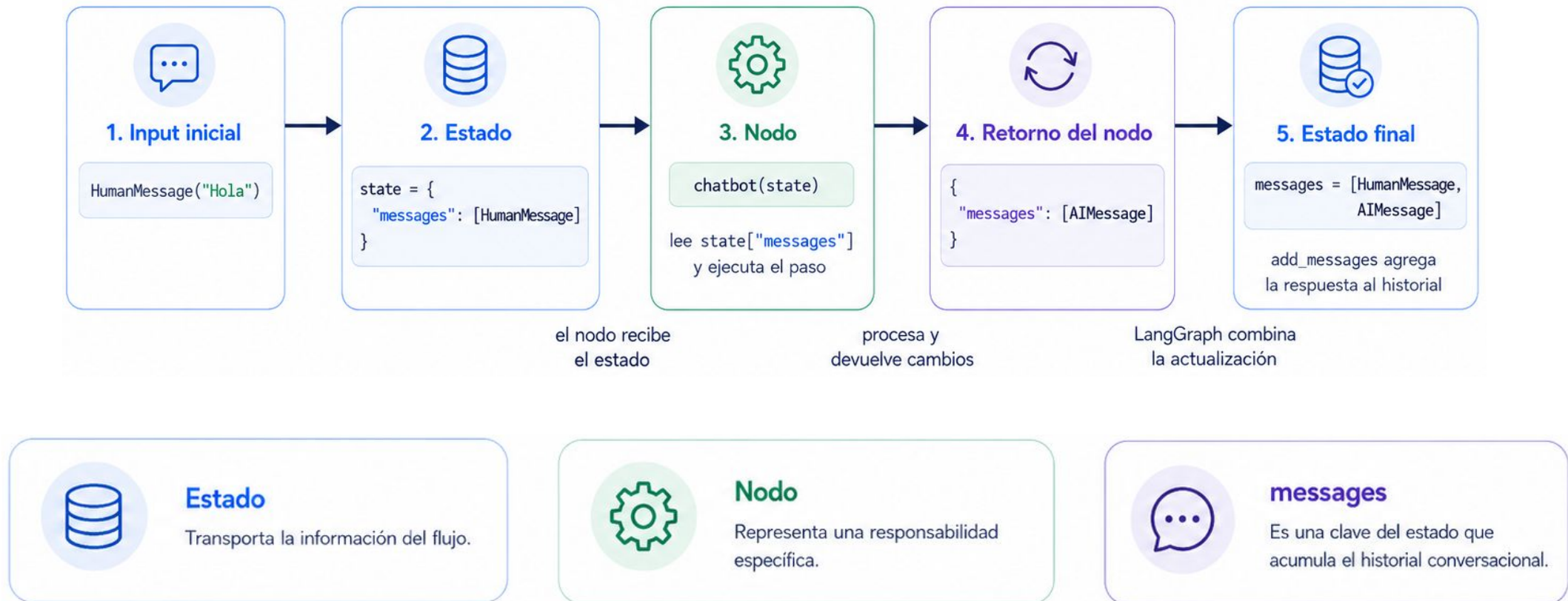
El estado final se conserva en el historial y se agrega el nuevo mensaje

Qué es un nodo

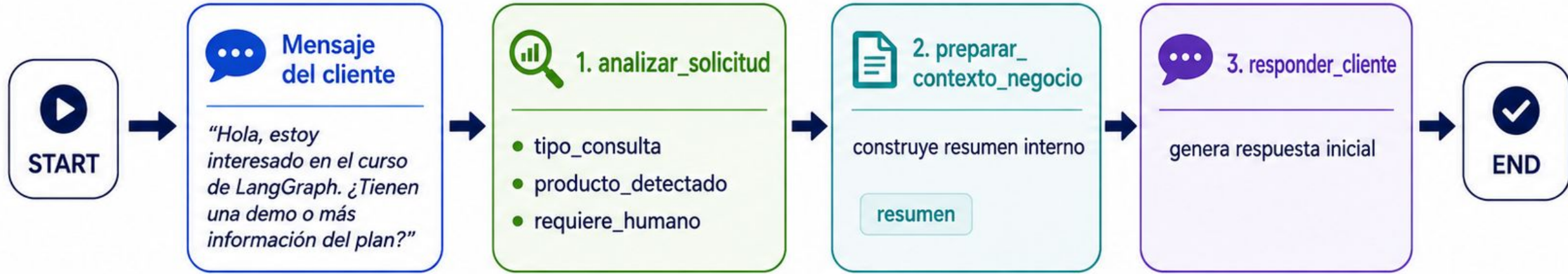
Un nodo es una función que recibe el estado actual y devuelve una actualización del estado.



Cómo interactúan estado, nodo y messages



Revisión del Notebook



 Cada nodo cumple una responsabilidad específica dentro del workflow.

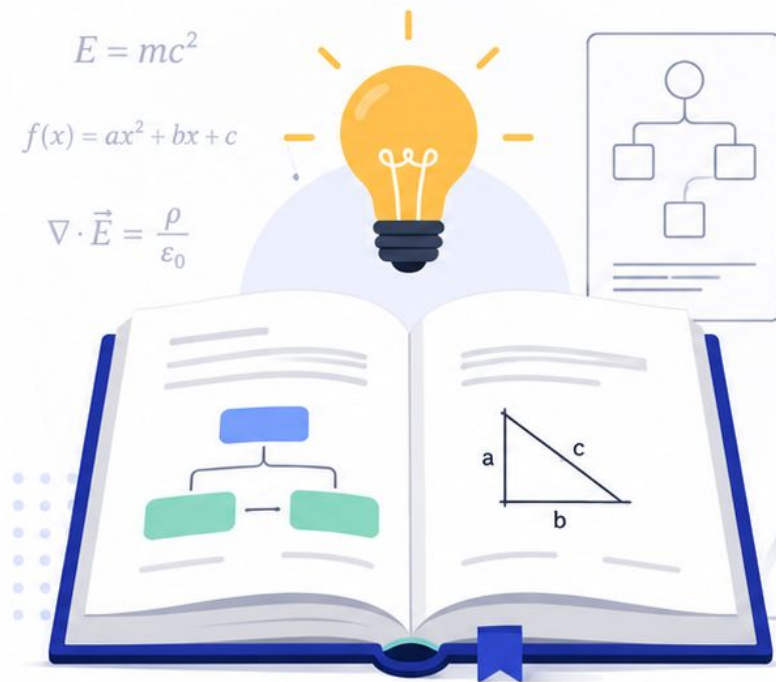


Resultado esperado

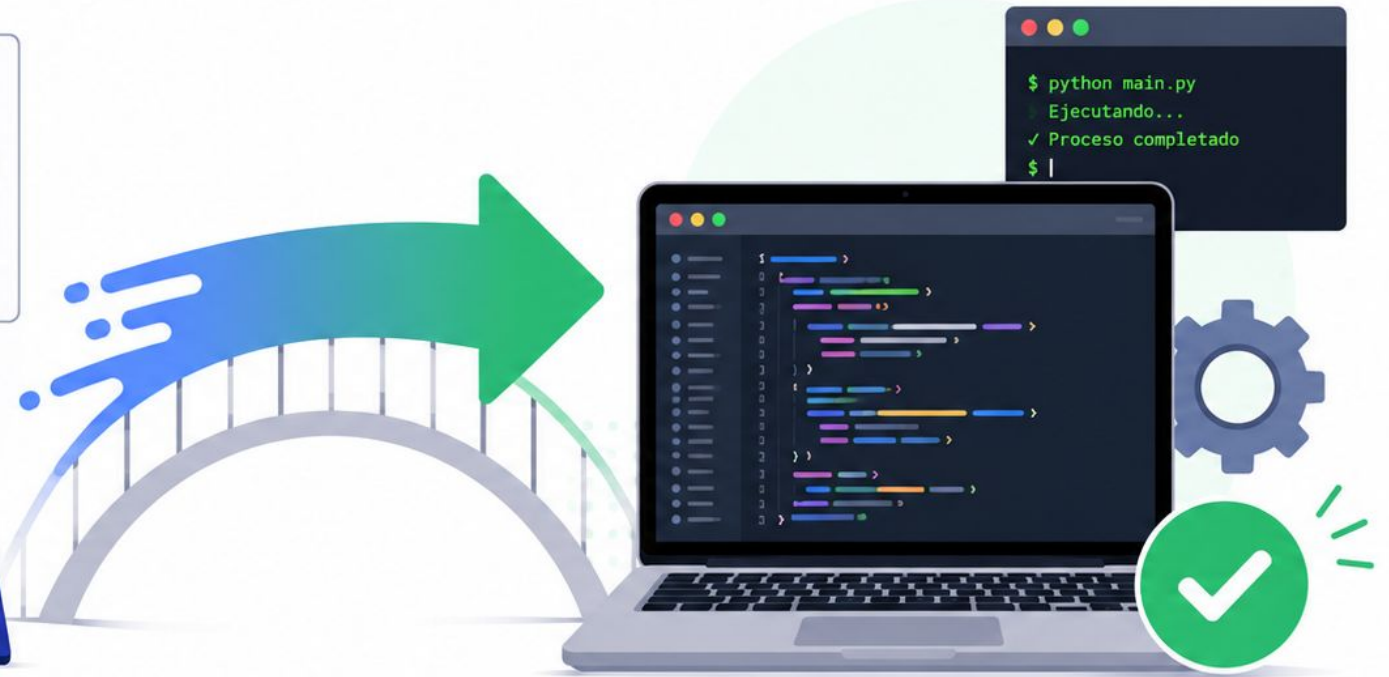
“ ¡Gracias por tu interés! Podemos compartirte más información sobre el curso y coordinar una demo inicial. ”

Revisión del Notebook

Teoría



Práctica



Conclusiones finales

En este capítulo vimos la base mínima de LangGraph: qué es un grafo, qué es el estado, qué es un nodo y cómo funciona el campo **messages** dentro del estado. Lo más importante es entender que LangGraph no ejecuta pasos aislados, sino que mueve estado entre nodos. En este primer ejemplo, ese estado contiene una lista de mensajes, y **add_messages** permite que las respuestas se vayan agregando al historial.